

AI Engineer Technical Homework

Task: Integration and Application Development

This homework task is designed to assess skills relevant to a Forward Deployed AI Engineer role, focusing on integration, application building, and core algorithmic problem-solving. Candidates are expected to demonstrate proficiency with standard Python libraries and tool integration, rather than extensive machine learning model training expertise.

General Remarks:

- You are encouraged to use all the AI-assisted development tools as you wish (Cursor, Claude Code, etc) except when writing tests. This assignment should take about 1 hour when these tools are used maximally.
- Part 1 and 2 are unrelated.

Final Delivery:

- Zip file representing a repo. It should contain the solution to both tasks.
- Repo should work out of the box and use [uv](#) with a *pyproject.toml*. *uv sync* should be the only command needed. Do not use *requirements.txt* or *pip*. It should be developed for either Python3.12 or Python3.13.
- All dependencies should be pinned to an exact version.
- Repo should have */tests* sub-directory with a single unit test for each task. It does not matter what the test is (please write the test yourself instead of using an LLM).
- It should have clear instructions as *readme.MD* including how to install, execute, and run tests.

Part 1: Named Entity Recognition Chatbot Integration

Develop a **single-turn**, [Pydantic AI](#)-based chatbot that can answer questions about named entities. Interface should be **command line** (no strict format requirement here) for example the PDF document input path and entity labels. The chatbot functionality in this exercise is on purpose left open. You can use your creativity.

Requirements

1. **NER Implementation:** Use the [Glinker library](#) to perform Named Entity Recognition on a given test document. You can choose the smallest NER model from glinker. Use [kreuzberg](#) for text extraction/OCR first. Save the detected named entities in the format you prefer (local DB or json etc).

2. **Pydantic Integration:** Define a Pydantic model for the expected chatbot output, ensuring the final response includes:
 - A conversational reply acknowledging the query.
 - A list of all unique named entities.
 - For each entity, its associated entity type/label.
3. **Chatbot Logic:** The chatbot should accept a natural language question (e.g., "How many mentions of Ukraine in the doc?") and respond based purely on the already detected entity DB/file (and not on the original text). Put some thought into this interface and feel free to propose your own ideas. The focus is on the correct extraction and structured output. Do not try to zero-shot the entity detection and question answering with an LLM in a single query. Chatbot does not need to be multi-turn (no need for history etc).

Part 2: Blob Bounding Box Generation

Given a binary semantic segmentation mask, develop an algorithm to identify individual object instances (blobs) and return a specifically defined rotated rectangle coordinates for each instance (details below).

Requirements

1. **Input:** A 2D NumPy array representing a binary mask. This mask contains multiple distinct, non-overlapping connected components (blobs).
2. **Task:** Implement a method to identify each blob. For each blob, calculate the rotated rectangle bounding box that will maximize the *Intersection-over-Union* of the blob and that mask defined by the rectangle itself (filled rectangle essentially).
3. **Output Format:** Write the results to a single CSV file named `blob_bounding_boxes.csv`. Each row must correspond to a unique blob instance and contain the following columns:

Column Name	Description	Example Value
blob_id	A unique identifier for the blob	1
center_x	X-coordinate of the box center	123.5
center_y	Y-coordinate of the box center	456.0
width	Width of the rotated bounding box	50.2
height	Height of the rotated bounding box	30.1

Column Name	Description	Example Value
angle	Rotation angle of the box in degrees (e.g., -90 to 90)	45.0



Example semantic segmentation mask